📄 جزوه سیستم عامل - فصل ۷: مثال‌های همگامی

(بخش چهارم و پایانی: همگامی در جاوا + جمع‌بندی کل فصل)

۱. همگامی در جاوا (Java Synchronization)

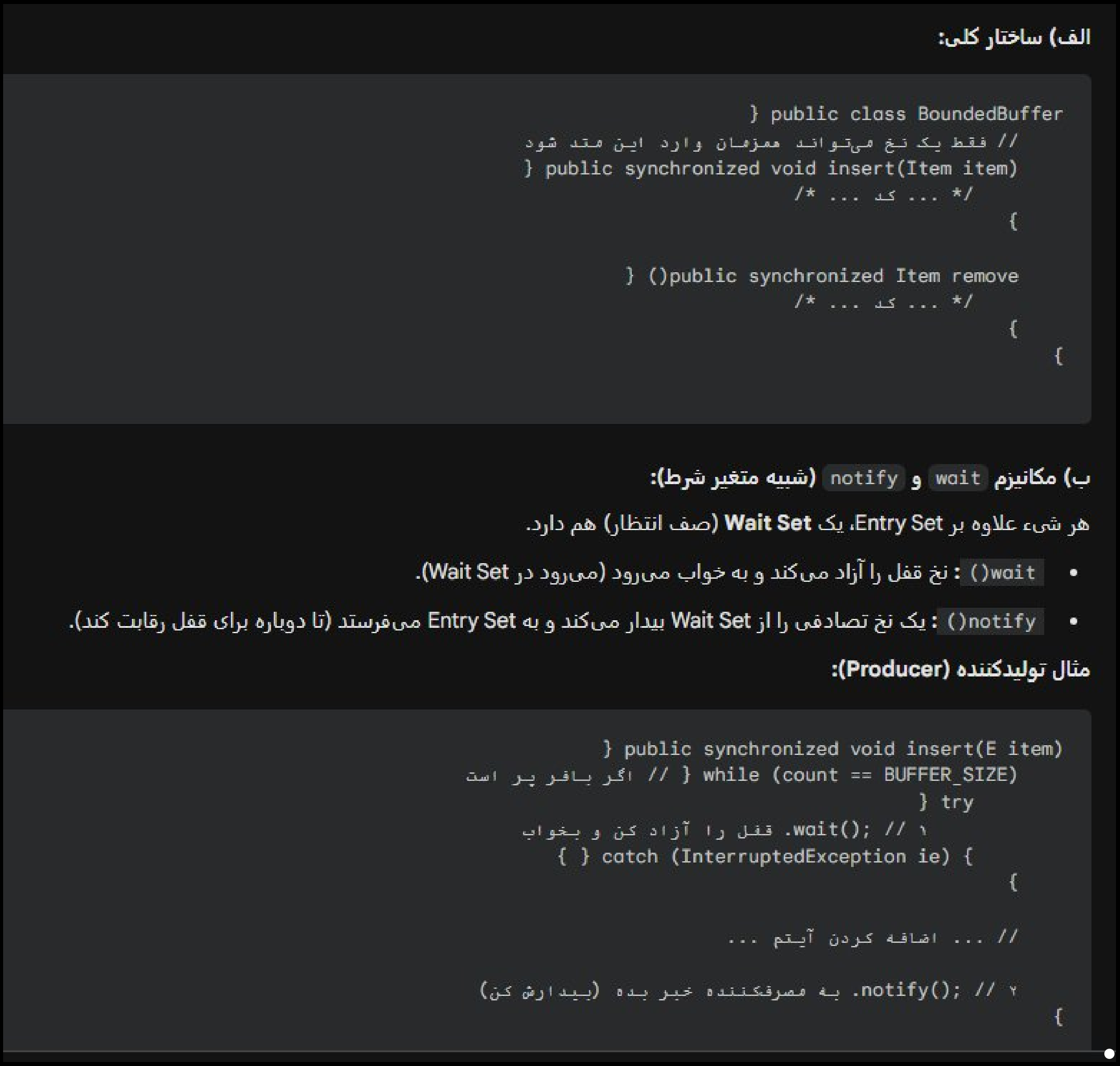
جاوا از همان ابتدا (برخلاف C که نیاز به کتابخانه دارد) پشتیبانی از "نخ" (Thread) و "همگامی" را در دل زبان خود داشته است. جاوا دو مدل همگامی دارد:

1. **مانیتورهای جاوا (Java Monitors):** روش قدیمی و ساده (با کلمه کلیدی synchronized).
2. **بسته java.util.concurrent:** روش‌های جدیدتر و پیشرفته (از نسخه ۱.۵ به بعد) شامل قفل‌ها، سمافورها و...

۲. مانیتورهای جاوا (روش synchronized)

هر شیء (Object) در جاوا یک **قفل (Lock)** داخلی دارد.

- وقتی یک متد را synchronized تعریف می‌کنید، نخ برای اجرای آن باید قفل آن شیء را بگیرد.
- اگر قفل دست نخ دیگری باشد، نخ جدید **مسدود (Block)** شده و در **Entry Set** (صف ورودی) قرار می‌گیرد.



الف) ساختار کلی:

```
} public class BoundedBuffer
   // فقط یک نخ می‌تواند همزمان وارد این متد شود
   } public synchronized void insert(Item item)
       /* ... کد ... */
   {

   } ()public synchronized Item remove
       /* ... کد ... */
   {
{
```

ب) مکانیزم `wait` و `notify` (شبیه متغیر شرط):

هر شیء علاوه بر Entry Set، یک **Wait Set** (صف انتظار) هم دارد.

- `wait()` : نخ قفل را آزاد می‌کند و به خواب می‌رود (می‌رود در Wait Set).
- `notify()` : یک نخ تصادفی را از Wait Set بیدار می‌کند و به Entry Set می‌فرستد (تا دوباره برای قفل رقابت کند).

مثال تولیدکننده (Producer):

```
} public synchronized void insert(E item)
   } while (count == BUFFER_SIZE) // اگر بافر پر است
       } try
           .wait(); // ۱ قفل را آزاد کن و بخواب
       { } catch (InterruptedException ie) {
   {

   // ... اضافه کردن آیتم ...

   .notify(); // ۲ به مصرفکننده خبر بده (بیدارش کن)
{
```

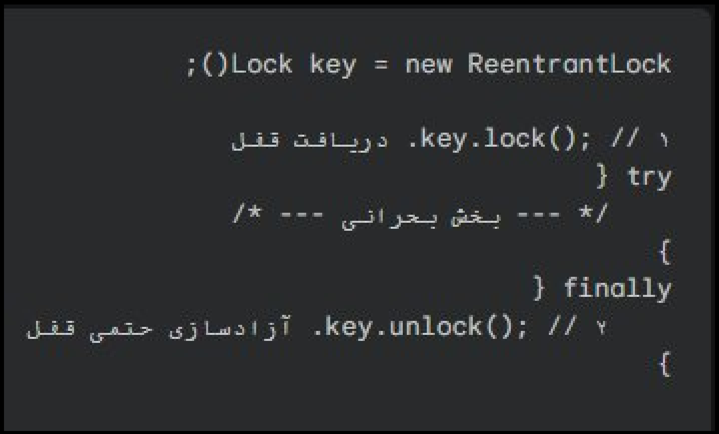
۳. قفل‌های چندمدخلی (Reentrant Locks)

چون synchronized گاهی محدودیت دارد (مثلاً نمی‌توانیم زمان انتظار تعیین کنیم یا قفل را در یک متد بگیریم و در متد دیگر آزاد کنیم)، جاوا کلاس ReentrantLock را معرفی کرد.

> **ویژگی Fairness (انصاف):** می‌توانیم تعیین کنیم که قفل به نخی برسد که بیشترین انتظار را کشیده (FIFO). در synchronized انتخاب تصادفی بود.

الگوریتم استفاده (بسیار مهم: الگوی try-finally):

حتماً باید آزادسازی قفل در بلاک finally باشد تا اگر خطایی رخ داد، قفل تا ابد گیر نکند.



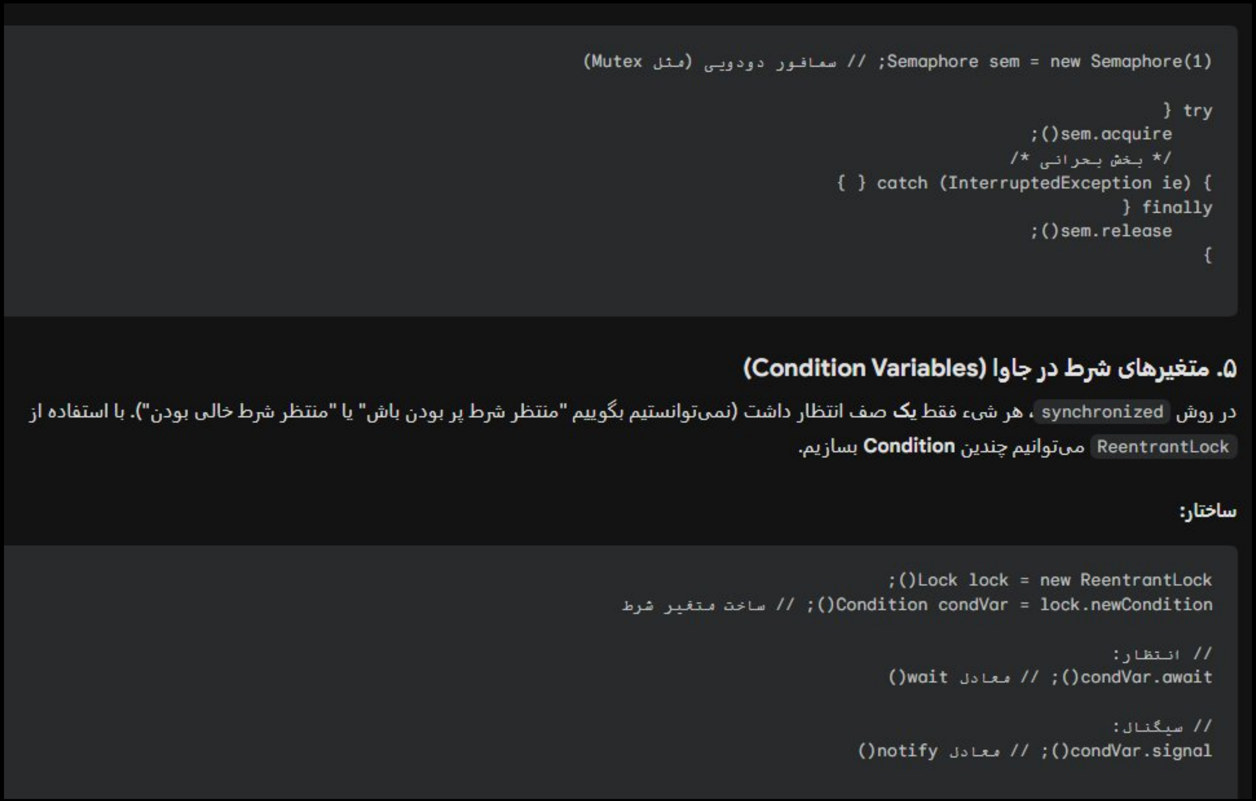
```
;()Lock key = new ReentrantLock

.key.lock(); // ۱ دریافت قفل
} try
/* --- بخش بحرانی --- */
{
} finally
.key.unlock(); // ۲ آزادسازی حتمی قفل
{
```

۴. سمافورها در جاوا

جاوا کلاس Semaphore را دارد که دقیقاً مثل سمافورهای سیستم‌عامل عمل می‌کند.

- سازنده: Semaphore(int value)
- متد کاهش (Wait): sem.acquire()
- متد افزایش (Signal): sem.release()

مثال:



```
(Mutex مثل) سمافور دودویی // ;Semaphore sem = new Semaphore(1)

} try
;()sem.acquire
/* بخش بحرانی */
{ } catch (InterruptedException ie) {
} finally
;()sem.release
{
```

۵. متغیرهای شرط در جاوا (Condition Variables)

در روش `synchronized`، هر شیء فقط **یک** صف انتظار داشت (نمی‌توانستیم بگوییم "منتظر شرط پر بودن باش" یا "منتظر شرط خالی بودن"). با استفاده از `ReentrantLock` می‌توانیم چندین **Condition** بسازیم.

ساختار:

```
;()Lock lock = new ReentrantLock
ساخت متغیر شرط // ;()Condition condVar = lock.newCondition

// انتظار:
()wait معادل // ;()condVar.await

// سیگنال:
()notify معادل // ;()condVar.signal
```

کاربرد در اولویت‌دهی (مثال doWork):

فرض کنید ۵ نخ داریم و می‌خواهیم به نوبت اجرا شوند (نخ ۰، بعد ۱، بعد ۲...).

- ۵ متغیر شرط (condVars[5]) می‌سازیم.
- هر نخ اگر نوبتش نبود، روی متغیر شرط خودش await می‌کند.
- وقتی کار نخی تمام شد، متغیر شرطِ نفر بعدی را signal می‌کند.

۶. 🌟 خلاصه و جمع‌بندی نهایی فصل ۷ 🌟

(این بخش را برای شب امتحان مرور کنید)

در این فصل یاد گرفتیم چطور از ابزارهای فصل ۶ در دنیای واقعی استفاده کنیم:

- **مسائل کلاسیک همگامی:**
  - **بافر محدود:** ۳ سمافور (mutex برای حفاظت، full/empty برای شمارش).
  - **خوانندگان-نویسندگان:** سمافور rw_mutex. خوانندگان می‌توانند همزمان وارد شوند، اما نویسنده باید تنها باشد. مشکل: قحطی‌زدگی نویسنده.
  - **فلاسفه خوش‌خوراک:** مشکل بن‌بست. راه حل: استفاده از مانیتور و چک کردن وضعیت دو همسایه قبل از برداشتن چوبک.
- **همگامی در سیستم‌عامل‌ها:**
  - **ویندوز:** در چندپردازنده از Spinlock استفاده می‌کند (نخ دارای قفل نباید Preempt شود). خارج از هسته از Dispatcher Objectها استفاده می‌کند.
  - **لینوکس:** ساده‌ترین ابزار atomic_t (اعداد اتمیک) است. از mutex_lock برای خوابیدن و Spinlock برای انتظار کوتاه استفاده می‌کند.
- **همگامی در APIها:**
  - **POSIX (زبان C/C++):** استاندارد جهانی. pthread_mutex (قفل)، sem_wait/post (سمافور) و pthread_cond (متغیر شرط).
  - **Java:**
    - **Monitors:** ساده، با کلمه synchronized و متدهای wait/notify.
    - **Lock API:** پیشرفته، با ReentrantLock و Condition.

پایان فصل هفتم. اکنون شما یک جزوه کامل و حرفه‌ای از تمام فصل ۷ دارید که تمام نکات ریز و درشت، کدها و مفاهیم را پوشش داده است. موفق باشید! ✌️